PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DUOC UC

DOCUMENTACIÓN TÉCNICA

Sistema de Gestión y Venta de Gas

"Distribuidora de Gas El Volcán"

Asignatura	DSY1104 — Desarrollo Fullstack II
Integrantes	Nicolás Arancibia Pablo Gómez Benjamín González
Profesor(a)	Hernán Saavedra
Versión	2.1

CHILLÁN · REGIÓN DE ÑUBLE · CHILE

Documentación — Versión 2 (El Volcán en React)

Explica qué se construyó en `Evaluacion 2/avance entrega`, cómo funciona cada parte y la lógica detrás de cada decisión. Complementa al `README.md` (en la raíz del proyecto), al `ERS_ACTUALIZADO.md` (requisitos) y al `DOCUMENTO_COBERTURA_TESTING.md` (pruebas).

---

1. Qué es y para qué

Aplicación **SPA** de la **Distribuidora de Gas El Volcán** construida con React. Reemplaza el sitio estático (HTML/CSS/JS) de la Versión 1 por una aplicación moderna, incorporando todo lo visto en la asignatura:

- Componentes y **props**.
- **Estado** (`useState`) y persistencia (`localStorage`).
- **React Router** (SPA con varias rutas y rutas anidadas).
- **Formularios controlados** con validación.
- **Diseño responsivo** con Bootstrap / React Bootstrap.
- **Pruebas unitarias** con Vitest + React Testing Library.

Datos de negocio reales: 14 productos de gas, precios residencial/comercial y el login del administrador (`admin@duoc.cl`).

---

2. Stack y por qué

Tecnología	Versión	Rol
Vite	8.3	Herramienta de desarrollo/build (`npm run dev`).
React / React DOM	19.2	Biblioteca de componentes (SPA).
react-router-dom	7.18	Rutas (`BrowserRouter` , `Routes` , `Route` , `Outlet` , `Link` , `useParams` , `useLocation` , `useNavigate`).
react-bootstrap	2.10	Componentes UI (`Navbar` , `Card` , `Button` , `Form` , `Alert` , `Badge` , `ListGroup` , `Table` , `Modal` , `Row` , `Col`).
bootstrap	5.3.8	CSS base (grillas y estilos). Importado en `main.jsx` .
bootstrap-icons	1.13	Íconos (`bi-*`) usados en el sitio y el admin.
oxlint	1.81	Linter (`npm run lint`).
Vitest	3.2	Pruebas unitarias (`npm test`), integrado con Vite.
@testing-library/react + jest-dom + user-event	16.3 / 6.6 / 14.6	Renderizado, matchers de DOM e interacción real del usuario.
jsdom	26.1	Navegador simulado (DOM + `localStorage`) para las pruebas.
@vitest/coverage-v8	3.2	Reporte de cobertura (`npm run test:coverage`).

Por qué este stack: es el mismo de la asignatura (ver `Semana react/mi-primer-react`). Para las pruebas, **Vitest es el framework indicado por el docente** para esta evaluación: reutiliza la configuración de Vite, corre sobre **jsdom** (sin abrir un navegador) y produce cobertura con el provider `v8` de Node.

---

3. Estructura

```
avance entrega/
├─ index.html                        → punto de montaje de la SPA (lang="es")
├─ vite.config.js                    → config de Vite (incluye el bloque `test` de Vitest)
├─ src/
│  ├─ main.jsx                       → arranca React y carga Bootstrap, íconos y CSS
│  ├─ setupTests.js                  → config global de Vitest (matchers de jest-dom)
│  ├─ estilos.css                    → estilos heredados de la Versión 1 (incluye .fondo)
│  ├─ index.css                      → estilos globales mínimos
│  ├─ App.jsx                        → rutas + estado global del carrito
│  ├─ datos/
│  │  └─ productos.js                → los 14 productos base
│  ├─ utils/
│  │  ├─ validaciones.js             → validaciones, REGIONES, CATEGORIAS, CLAVES, helpers
│  │  ├─ carrito.js                  → lógica pura del carrito
│  │  └─ productos.js                → catálogo real (ediciones + creados)
│  ├─ components/
│  │  ├─ LayoutPublico.jsx           → layout de la parte pública
│  │  ├─ Navbar.jsx                  → barra de navegación
│  │  ├─ Footer.jsx                  → pie de página
│  │  ├─ BotonTienda.jsx             → botón flotante a Tiendas
│  │  └─ Producto.jsx                → tarjeta de producto
│  ├─ pages/
│  │  ├─ Inicio.jsx
│  │  ├─ Productos.jsx
│  │  ├─ DetalleProducto.jsx
│  │  ├─ Carrito.jsx
│  │  ├─ Ventas.jsx                  → checkout
│  │  ├─ Login.jsx
│  │  ├─ Registro.jsx
│  │  ├─ Nosotros.jsx
│  │  ├─ Consejos.jsx
│  │  ├─ Tiendas.jsx
│  │  ├─ Contacto.jsx
│  │  └─ admin/
│  │     ├─ AdminLayout.jsx
│  │     ├─ Dashboard.jsx
│  │     ├─ Ordenes.jsx              → panel de órdenes (datos de ejemplo)
│  │     ├─ Clientes.jsx             → panel de clientes (datos de ejemplo)
│  │     ├─ Reportes.jsx             → panel de reportes (datos de ejemplo)
│  │     ├─ Inventario.jsx
│  │     ├─ ListaUsuarios.jsx
│  │     └─ NuevoUsuario.jsx
│  └─ (las pruebas **.test.js(x)** viven junto a cada módulo: utils/, components/, pages/)
```

```
└─ public/
   └─ IMAGENES/                    → imágenes de los productos y del sitio
```

---

4. Rutas de la aplicación

Definidas en `src/App.jsx` con **rutas anidadas**: las públicas comparten `LayoutPublico` y las de administración comparten `AdminLayout` (cada layout renderiza su contenido con `<Outlet />`).

Ruta	Componente	Layout	Descripción
`/`	`Inicio`	Público	Portada
`/productos`	`Productos`	Público	Catálogo (con `?categoria=`)
`/producto/:id`	`DetalleProducto`	Público	Ficha de producto
`/carrito`	`Carrito`	Público	Carrito
`/ventas`	`Ventas`	Público	Checkout
`/login`	`Login`	Público	Acceso
`/registro`	`Registro`	Público	Crear cuenta
`/nosotros`	`Nosotros`	Público	Historia
`/consejos`	`Consejos`	Público	Consejos de seguridad
`/tiendas`	`Tiendas`	Público	Sucursales
`/contacto`	`Contacto`	Público	Formulario de contacto
`/admin`	`Dashboard`	Admin	Panel (índice)
`/admin/inventario`	`Inventario`	Admin	CRUD de productos
`/admin/usuarios`	`ListaUsuarios`	Admin	Usuarios y roles
`/admin/usuarios/nuevo`	`NuevoUsuario`	Admin	Crear usuario

---

5. Estado global y persistencia

El **carrito** es el único estado de aplicación; vive en `App` (patrón **"lifting state up"**) porque lo comparten `Productos`, `DetalleProducto`, `Carrito` y `Ventas`.

```
 const [carrito, setCarrito] = useState(() => {
  try { return JSON.parse(localStorage.getItem(CLAVES.carrito)) || [] }
  catch { return [] }
})

useEffect(() => {
  localStorage.setItem(CLAVES.carrito, JSON.stringify(carrito))
}, [carrito])
```

- El valor inicial se **lee de** `localStorage` con la función inicializadora del `useState` (por eso `F5` no lo pierde).
- Cada cambio se **persiste** con `useEffect`.
- Las operaciones (`agregar`, `incrementar`, `decrementar`, `eliminar`, `vaciar`) delegan en las funciones puras de `utils/carrito.js`, y `totalItems` se calcula con `contarItems`.
- `App` le pasa el carrito y los callbacks por **props** a `Carrito` / `Ventas`, y `totalItems` a `Navbar`.

Claves de `localStorage` (constante `CLAVES` en `utils/validaciones.js`):

Clave	Contenido
`carrito`	Arreglo `[{ id, cantidad }]`
`usuarios`	Usuarios registrados / creados por el admin
`sesion`	Usuario con sesión activa (sin contraseña)
`productos`	Productos creados desde el panel admin
`edicionesProductos`	Cambios del admin sobre los productos base
`ordenes`	Órdenes generadas en el checkout
`solicitudes`	Mensajes enviados por el formulario de contacto

---

6. Utilidades (lógica pura y testable)

Se extrajeron a `src/utils/` para reutilizarlas en varias páginas **y** poder probarlas de forma aislada.

6.1 `utils/validaciones.js`

Contiene datos compartidos (`REGIONES`, `CATEGORIAS`, `ADMIN_DEFAULT`, `CLAVES`) y funciones puras:

- `validarRun`, `validarTelefono`, `validarCorreo`, `correoDominioPermitido`, `formatearPrecio`.
- `validarFormularioRegistro(datos)` y `validarFormularioContacto(datos)`: devuelven **el primer mensaje de error** o `''` si todo está correcto (centralizan las reglas que antes estaban dispersas en `java.js`).
- `leerLista(clave)` / `leerObjeto(clave)`: leen `localStorage` de forma **tolerante a errores** (JSON corrupto devuelve `[]` o `null` en vez de romper la app).

6.2 `utils/carrito.js`

Funciones **puras** (reciben el carrito y devuelven uno **nuevo**, sin mutar), fáciles de testear:

- `agregarItem`, `incrementarItem`, `decrementarItem` (elimina al llegar a 0), `eliminarItem`, `contarItems`.
- `obtenerItemsCompletos(carrito, productos)`: une cada línea con su producto y descarta ids inexistentes.
- `calcularTotal(carrito, productos, tipo)`: suma `precio × cantidad` según `residencial` o `comercial`.

6.3 `utils/productos.js`

Construye el **catálogo real** combinando datos:

- `aplicarEdiciones(base, ediciones)`: sobreescribe solo los campos editados por el admin.
- `catalogoActual()`: catálogo base + ediciones (lo que ve el cliente).
- `productosConCreados()`: catálogo + productos creados desde el admin, marcados con `base: true/false`.

> **Por qué importa**: son el "seam" (punto único de cambio). Cuando existan los microservicios, estas funciones se reemplazan por `fetch()` y las páginas no cambian.

---

7. Componentes reutilizables

7.1 `components/LayoutPublico.jsx`

Layout de la parte pública. Renderiza `BotonTienda`, `Navbar`, el `<Outlet />` (la página actual) y `Footer`. Gracias a las **rutas anidadas**, no repetimos el menú ni el pie en cada página.

7.2 `components/Navbar.jsx`

Barra de navegación de react-bootstrap (`Navbar as BootstrapNavbar` para no chocar con el nombre propio). Recibe `totalItems` **por props** y lo muestra en un `Badge` sobre el enlace "Carrito". `expand="lg"` la hace colapsable (responsiva). Enlaces: Inicio, Productos, Nosotros, Consejos, Tiendas, Contacto, Carrito, Login y Registro.

7.3 `components/Producto.jsx`

Tarjeta de producto (componente **de presentación**). Recibe props (`id`, `nombre`, `descripcion`, `residencial`, `imagen`) y un callback `onAgregar`. **No** sabe de datos: solo muestra y avisa.

- "Agregar" → `props.onAgregar(props.id)` (comunicación **hijo → padre**).
- "Ver detalle" → `Link to={/producto/${props.id}}`.

7.4 `components/Footer.jsx` y `components/BotonTienda.jsx`

- `Footer`: logo, datos de contacto y créditos.
- `BotonTienda`: botón flotante con imagen que enlaza a `/tiendas`, con `aria-label` accesible.

8. Páginas públicas

8.1 `Inicio.jsx`

Componente de presentación: título, descripción y botón `as={Link}` a `/productos`.

8.2 `Productos.jsx` — catálogo + filtro

```
const categoria = new URLSearchParams(useLocation().search).get('categoria')
const productos = catalogoActual()
const lista = categoria ? productos.filter((p) => p.categoria === categoria) : productos
```

Usa `useLocation` + `URLSearchParams` (funcionalidad "parámetros de búsqueda" de la guía). Los botones de categoría navegan a `/productos?categoria=...`; las tarjetas se generan con `.map()` y `key={p.id}`, en grilla responsiva.

8.3 `DetalleProducto.jsx` — ficha

```
const { id } = useParams()
const producto = catalogoActual().find((p) => p.id === id)
```

`useParams` extrae `:id`; si el producto no existe muestra un `Alert` "Producto no encontrado" (caso real: URL tecleada a mano). Muestra imagen, descripción, categoría, código, precios y stock, con botón "Agregar".

8.4 `Carrito.jsx` — resumen y total

```
const items = obtenerItemsCompletos(carrito, catalogoActual())
const total = calcularTotal(carrito, catalogoActual())
```

Muestra cada línea con `+` / `-` /Quitar (los callbacks vienen de `App`), subtotales y total, y el botón "Finalizar compra" que lleva al checkout (`/ventas`). Si está vacío, muestra un `Alert` y un botón a Productos.

8.5 `Ventas.jsx` — checkout

- Requiere **sesión iniciada**; si no hay, invita a iniciar sesión/registrarse (comparando con `localStorage.sesion`).
- Toma dirección y comuna de la sesión (se pueden editar) y pide **método de pago** (Efectivo / Tarjeta).
- Simula una **pasarela de pago** (sin backend): Efectivo siempre aprueba (se paga en la entrega); Tarjeta puede ser rechazada (~20%), mostrando la vista **"No se pudo realizar el pago"** — en ese caso **no se guarda la orden** y se puede **reintentar** sin perder los datos.
- Al aprobar, **genera una orden** (`ORD-######`) con fecha, cliente, ítems y total, la guarda en `localStorage.ordenes` y **vacía el carrito** (`vaciar`).
- Muestra una confirmación con el número de orden y el total.

8.6 `Login.jsx` — acceso y sesión

Formulario controlado. Valida `@` en el correo y largo de contraseña; busca en `localStorage.usuarios` + el `ADMIN_DEFAULT`. Si coincide, guarda la **sesión** (sin contraseña) en `localStorage.sesion` y navega (el administrador va a `/admin`). Si ya hay sesión, muestra los datos y un botón "Cerrar sesión".

8.7 `Registro.jsx` — creación de cuenta

Formulario controlado por campo. Usa `validarFormularioRegistro` (RUN 7–9 dígitos, correo `@duoc.cl` / `@profesor.duoc.cl` / `@gmail.com`, teléfono, dirección, región/comuna, contraseña 4–10, términos) y verifica que el correo no esté repetido. Guarda el usuario con `rol: 'Cliente'` y navega a `/login`. Usa **selectores dependientes** Región → Comuna (`REGIONES.find(...)`).

8.8 `Nosotros.jsx` — historia con blogs expandibles

Dos secciones ("blogs") con resumen y un botón "Ver Más"/"Ver Menos" que **expande/contrae** el contenido usando `useState` (estado local por blog). Esta página se usa para probar el **estado de un componente**.

8.9 `Consejos.jsx` — seguridad del gas

Secciones informativas: instalación correcta, consejos rápidos (íconos), video de YouTube embebido y pasos a seguir ante una fuga. Incluye un CTA al catálogo.

8.10 `Tiendas.jsx` — sucursales

Tarjetas de las sucursales (Casa Matriz Chillán, Chillán Viejo, Bulnes, Quillón, San Ignacio) con dirección, teléfono, horario y un **mapa** referencial.

8.11 `Contacto.jsx` — formulario

Valida con `validarFormularioContacto` (nombre, correo, teléfono, asunto y mensaje) y **guarda la solicitud** en `localStorage.solicitudes`, mostrando confirmación.

---

9. Panel de administración

9.1 `admin/AdminLayout.jsx`

Layout propio del panel: menú lateral (`NavLink`) con Dashboard, Inventario y Empleados, un acceso a "+Profile" (Nuevo usuario), el nombre del usuario de la sesión y el botón **Cerrar sesión** (elimina `localStorage.sesion` y vuelve a `/login`). En móvil el menú se colapsa. Los ítems **Órdenes**, **Reportes** y **Clientes** ya cuentan con vistas propias (integradas por el compañero; ver 9.6).

9.2 `admin/Dashboard.jsx`

Indicadores (ventas del día, órdenes en ruta, clientes nuevos), tabla de últimas órdenes con estados (badges) y **alertas de stock crítico** calculadas con `catalogoActual()`.

9.3 `admin/Inventario.jsx` — CRUD de productos

- Lista `productosConCreados()` (base + creados) en una tabla, resaltando **stock crítico**.

- **Crear**: valida código único, nombre, precios, stock entero y categoría; guarda en `localStorage.productos`.
- **Editar**: guarda cambios en `localStorage.edicionesProductos` para los base, sin sobrescribir el catálogo original (por eso `aplicarEdiciones` conserva los campos no editados).
- **Eliminar**: solo los productos creados (los base solo se editan).

9.4 `admin/ListaUsuarios.jsx`

Lista `ADMIN_DEFAULT` + usuarios de `localStorage.usuarios`, con **filtro por rol** (Todos, Administradores, Clientes, Vendedores) y botón para crear un usuario nuevo.

9.5 `admin/NuevoUsuario.jsx`

Formulario para crear usuarios con rol (`Administrador` / `Cliente` / `Vendedor`), reutilizando `validarRun`, `validarTelefono`, `correoDominioPermitido` y los selectores Región → Comuna. Guarda en `localStorage.usuarios`.

9.6 Vistas integradas por el compañero (maquetas con datos de ejemplo)

- `admin/Ordenes.jsx` (`/admin/ordenes`): KPI de órdenes del mes, tabla de órdenes con estados (badges) y distribución por comuna.
- `admin/Clientes.jsx` (`/admin/clientes`): KPI de clientes, listado con tipo y últimas compras, mejores clientes y distribución por comuna.
- `admin/Reportes.jsx` (`/admin/reportes`): KPI de ingresos, ventas mensuales (barras), productos más vendidos, ventas por categoría y tipo de cliente, y reportes generados.

Estas vistas usan contenido estático de ejemplo (no dependen de `localStorage`) y reutilizan clases de Bootstrap + íconos `bi-*`, coherentes con el resto del panel. Al conectar el futuro backend se reemplazarán los arreglos constantes por datos de la API.

---

10. Pruebas unitarias (Vitest)

Configuración (`vite.config.js`):

- Bloque `test` con `environment: 'jsdom'`, `setupFiles: './src/setupTests.js'`, `globals: true` y `css: false`.
- `esbuild: { jsx: 'automatic' }` para que los tests `.jsx` usen el runtime automático de React 19 sin importar React.
- Cobertura con el **provider** `v8` (`@vitest/coverage-v8`) que reporta en consola, `coverage/index.html` y `coverage/lcov.info`.

Arranque (`src/setupTests.js`): importa `@testing-library/jest-dom` para los matchers de DOM (`toBeInTheDocument`, `toHaveTextContent`, `toHaveValue`, `toHaveAttribute`).

Las 10 pruebas (agrupadas por cohesión, ver `DOCUMENTO_COBERTURA_TESTING.md`):

#	Test	Qué verifica
1	`validaciones.test.js`	RUN, teléfono, correo, dominio permitido y `formatearPrecio`

#	Test	Qué verifica
2	`validaciones.test.js`	`validarFormularioRegistro` + `validarFormularioContacto` (primer error) y `leerLista` / `leerObjeto` (JSON corrupto)
3	`carrito.test.js`	`agregarItem` / `incrementarItem` acumulan sin mutar el original (carrito)
4	`carrito.test.js`	`decrementarItem` (elimina en 0), `eliminarItem` y `contarItems`
5	`carrito.test.js`	`obtenerItemsCompletos` (descarta ids) y `calcularTotal` (residencial/comercial) — compras
6	`productos.test.js`	`aplicarEdiciones`, `catalogoActual` y `productosConCreados` — CRUD
7	`Producto.test.jsx`	props en el DOM y callback `onAgregar` (mock `vi.fn()`)
8	`Navbar.test.jsx`	enlaces (`href`) y prop `totalItems` en el badge
9	`Nosotros.test.jsx`	estado `useState`: expandir/contraer el blog
10	`Login.test.jsx`	formulario interactivo: los campos se asocian por `label`, el usuario escribe (`user-event`) y el correo inválido muestra error

Cobertura: 100 % sentencias · 100 % ramas · 100 % funciones · 100 % líneas (módulos bajo prueba). Las ramas no cubiertas son defensivas (validaciones alternativas, respaldos `|| 0`, ediciones parciales) y las del login con credenciales válidas (se validan manualmente en el panel admin).

---

11. Decisiones de diseño

Decisión	Por qué
Carrito en `App`, no en cada página	Estado compartido se centraliza en el padre ("lifting state up").
Lógica en `utils/` (pura)	Se reutiliza en varias páginas y se prueba de forma aislada.
Persistencia con `localStorage`	No hay backend todavía; igual que la Versión 1, no se pierde al `F5`.
Datos en `datos/productos.js` + `utils/productos.js`	Mismos 14 productos validados antes; un único punto de cambio para la futura API.
Rutas anidadas + `LayoutPublico` / `AdminLayout`	Evita repetir el menú/pie y separa claramente las dos zonas.
Ediciones del admin separadas de los datos base	Los productos base no se sobrescriben (`aplicarEdiciones` conserva lo no editado).
Precios con `toLocaleString('es-CL')` / `formatearPrecio`	Formato de moneda chilena, consistente con la entrega anterior.
Botón `-` que elimina el ítem en 0	La cantidad nunca puede quedar negativa.
10 pruebas agrupando funciones cohesivas	Cumple el requisito de **10 pruebas** maximizando cobertura.

---

12. Conceptos del curso que se ven aquí

- Componentes y **props** (padre → hijo).
- **Eventos y callbacks** (hijo → padre con `onAgregar`).
- **Estado** con `useState` y persistencia con `useEffect` + `localStorage`.
- **React Router**: `BrowserRouter`, `Routes`, `Route`, `Outlet` (rutas anidadas), `Link`, `useParams`, `useLocation`, `useNavigate`.
- **Formularios controlados** y validación (`preventDefault`).
- **Renderizado condicional** (`{error && <Alert/>}`) y **listas** con `.map()` + `key`.
- **Atomic Design** (componentes pequeños reutilizables → páginas).
- **Diseño responsivo** con grillas de Bootstrap.
- **Pruebas unitarias** (Vitest + React Testing Library) de funciones puras y de componentes (jest-dom + `vi.fn()`).

---

13. Cómo correrlo

```
 npm.cmd install     # la primera vez
npm.cmd run dev     # desarrollo → http://localhost:5173
npm run lint        # oxlint: 0 errores
npm run build       # producción → carpeta dist/
npm test            # 10 pruebas (una sola vez)
npm run test:watch  # pruebas en modo vigilancia
npm run test:coverage # pruebas + cobertura en coverage/
```

> En PowerShell usar `npm.cmd` (la política de ejecución puede bloquear `npm.ps1`). Las pruebas corren en **jsdom** (DOM simulado de Node), sin necesidad de abrir un navegador.

Credenciales de prueba (admin): `admin@duoc.cl` / `Admin123`.

---

14. Próximos pasos

- Backend con **microservicios Spring Boot** (API REST/JSON) y **MySQL**: reemplazar `datos/productos.js` y `localStorage` por llamadas `fetch()`.
- **Autenticación con tokens** y **RBAC** real (roles Operadora y Repartidor), protegiendo las rutas del panel.
- **Seguimiento de pedidos con mapas** (Leaflet / Google Maps).
- **React Hook Form** (`react-hook-form`) para los formularios (guía 2.2.2).

- **Despliegue en AWS EC2 con Docker + Nginx** (guía 2.1.2).